

```
#!/usr/bin/env python3
```

```
"""
```

```
06_provenance_gate.py - Stray-paste / fabrication / duplication gate.
```

Fails (exit 1) if it finds:

1. AI-session transcript artifacts leaking into data/docs files (e.g. "Model: <name>", "--- User ---", "<citation src=...>").
2. County->rating-area mappings hardcoded in more than one .py file, or hardcoded in .py at all while the JSON SSOT exists.
3. Citations marked excluded/fabricated in data/statutes_2026.json resurfacing elsewhere as if they were live (documenting files exempt).

This does NOT verify facts against primary sources. A clean run means "nothing obviously contaminated the repo" - NOT "every number is confirmed."

Usage:

```
python3 06_provenance_gate.py [--json] [--selftest]
```

```
"""
```

```
import json
```

```
import re
```

```
import sys
```

```
from pathlib import Path
```

```
# -----
```

```
# ROOT RESOLUTION (fix #2): walk up to a landmark instead of assuming layout
```

```
# -----
```

```
def find_root() -> Path:
```

```
    start = Path(__file__).resolve().parent
```

```
    for candidate in (Path(__file__).resolve().parent, *Path(__file__).resolve().parents):
```

```
        if (candidate / "data" / "statutes_2026.json").exists() or (candidate / ".git").exists():
```

```
            return candidate
```

```
    return Path(__file__).resolve().parent
```

```
ROOT = find_root()
```

```
FAILURES = [] # severity: "block"
```

```
WARNINGS = [] # severity: "warn"
```

```
AI_ARTIFACT_PATTERNS = [
```

```
    (r"^\\s*Model:\\s*\\S+", "model-transcript-header"),
```

```
    (r"^\\s*-{2,}\\s*User\\s*-{2,}", "user-divider"),
```

```
    (r"^\\s*-{2,}\\s*Assistant\\s*-{2,}", "assistant-divider"),
```

```
    (r"<citation\\s+src=", "citation-tag"),
```

```
    (r"\\bas an AI (language )?model\\b", "ai-disclaimer"),
```

```
]
```

```
# Files that legitimately CONTAIN excluded citations because their job is  
# to document the exclusions. Short, explicit, policy-statement (fix #3).
```

```
EXEMPT_DOCUMENTING = {
```

```
    "docs/source_reliability.md",
```

```
    "data/statutes_2026.json",
```

```
}
```

```

SKIP_DIRS = {".git", "node_modules", "__pycache__", ".venv", "tests"}
SCAN_EXTS = {".txt", ".md", ".json", ".csv", ".py"} # widened scan surface

# This gate's own source documents the patterns it looks for (docstring,
# pattern list) - without this exclusion it flags itself on every run.
SELF_PATH = Path(__file__).resolve()

SSOT_JSON = Path("data/public/pa_county_fips_crosswalk.json")

# Reference set for duplicate-crosswalk detection ONLY - not consumed as
# the operational SSOT by any pipeline code, just used here to recognize
# "this file embeds the PA county list" regardless of structure (dict-by
# -county, dict-by-area-with-list, or county->FIPS). Sourced from the
# verified data/rating_areas_2026.csv (67 counties, CMS-matched).
PA_COUNTIES_REFERENCE = {
    "Adams", "Allegheny", "Armstrong", "Beaver", "Bedford", "Berks", "Blair",
    "Bradford", "Bucks", "Butler", "Cambria", "Cameron", "Carbon", "Centre",
    "Chester", "Clarion", "Clearfield", "Clinton", "Columbia", "Crawford",
    "Cumberland", "Dauphin", "Delaware", "Elk", "Erie", "Fayette", "Forest",
    "Franklin", "Fulton", "Greene", "Huntingdon", "Indiana", "Jefferson",
    "Juniata", "Lackawanna", "Lancaster", "Lawrence", "Lebanon", "Lehigh",
    "Luzerne", "Lycoming", "McKean", "Mercer", "Mifflin", "Monroe",
    "Montgomery", "Montour", "Northampton", "Northumberland", "Perry",
    "Philadelphia", "Pike", "Potter", "Schuylkill", "Snyder", "Somerset",
    "Sullivan", "Susquehanna", "Tioga", "Union", "Venango", "Warren",
    "Washington", "Wayne", "Westmoreland", "Wyoming", "York",
}

def iter_files(exts):
    for path in ROOT.rglob("*"):
        if not path.is_file():
            continue
        if any(part in SKIP_DIRS for part in path.parts):
            continue
        if path.suffix.lower() in exts:
            yield path

def read(path):
    try:
        return path.read_text(encoding="utf-8", errors="ignore")
    except Exception:
        return None

def rel(path):
    return path.relative_to(ROOT).as_posix()

# -----
def scan_ai_artifacts():
    for path in iter_files(SCAN_EXTS):
        if path.resolve() == SELF_PATH:
            continue

```

```

text = read(path)
if text is None:
    continue
for pat, label in AI_ARTIFACT_PATTERNS:
    m = re.search(pat, text, re.MULTILINE | re.IGNORECASE)
    if m:
        snippet = text[max(0, m.start() - 20):m.end() + 40].replace("\n", "\\n")
        FAILURES.append(
            f"[AI-ARTIFACT:{label}] {rel(path)}: matches {pat}/ "
            f"near: ...{snippet}..."
        )

```

```

def scan_crosswalk_ssot():

```

```

    """Detect by county-NAME overlap against PA_COUNTIES_REFERENCE, not by
    FIPS literals - format-invariant across {name: area}, {area: [names]},
    and {name: fips} styles, which is what this repo's real files actually
    use (verified against the two known duplicates: neither embeds FIPS
    codes as {"fips": ..., "name": ...} pairs; one has no FIPS at all).

```

```

    Rules:

```

- (a) no two .py files may both embed 40+ of the reference counties;
- (b) if the JSON SSOT exists, no .py file should embed 40+ at all -
 it should import from the JSON instead.

```

    """

```

```

    hardcoders = {}

```

```

    for path in iter_files({"py"}):

```

```

        if path.resolve() == SELF_PATH:
            continue

```

```

        text = read(path)

```

```

        if text is None:
            continue

```

```

        found = set(re.findall(r'"([A-Z][a-zA-Z]+)"', text)) & PA_COUNTIES_REFERENCE

```

```

        if len(found) >= 40:
            hardcoders[path] = found

```

```

    paths = sorted(hardcoders)

```

```

    for i in range(len(paths)):

```

```

        for j in range(i + 1, len(paths)):

```

```

            overlap = hardcoders[paths[i]] & hardcoders[paths[j]]

```

```

            if len(overlap) >= 40:

```

```

                FAILURES.append(

```

```

                    f"[DUPLICATE-SSOT] {rel(paths[i])} and {rel(paths[j])} both "

```

```

                    f"embed the PA county list ({len(overlap)} shared counties). "

```

```

                    f"One file should import from the SSOT; the other is a stale copy."

```

```

                )

```

```

    if (ROOT / "data" / "public" / "pa_county_fips_crosswalk.json").exists():

```

```

        for p in paths:

```

```

            WARNINGS.append(

```

```

                f"[SSOT-BYPASS] {rel(p)} embeds {len(hardcoders[p])} PA counties "

```

```

                f"while data/public/pa_county_fips_crosswalk.json exists. "

```

```

                f"Should be imported, not embedded."

```

```

            )

```

```

def scan_excluded_citations_resurfacing():
    statutes_file = ROOT / "data" / "statutes_2026.json"
    if not statutes_file.exists():
        WARNINGS.append(
            "[NO-STATUTES-FILE] data/statutes_2026.json not found - "
            "excluded-citation check skipped. Seed it so this gate can run."
        )
    return

try:
    data = json.loads(statutes_file.read_text(encoding="utf-8"))
except Exception as e:
    FAILURES.append(f"[BAD-JSON] {rel(statutes_file)}: {e}")
    return

excluded = [c for item in data.get("excluded_citations", [])
             if (c := item.get("citation"))]
for path in iter_files(SCAN_EXTS):
    if path == statutes_file or rel(path) in EXEMPT_DOCUMENTING:
        continue
    text = read(path)
    if text is None:
        continue
    for citation in excluded:
        if citation in text:
            FAILURES.append(
                f"[EXCLUDED-CITATION-RESURFACED] {rel(path)}: contains "
                f"'{citation}', already marked excluded/fabricated in "
                f"data/statutes_2026.json."
            )

def report(as_json=False):
    payload = {
        "gate": "06_provenance_gate",
        "status": "FAILED" if FAILURES else ("WARNED" if WARNINGS else "PASSED"),
        "blocking": FAILURES,
        "warnings": WARNINGS,
        "caveat": "Clean run = no known contamination patterns. "
                  "It does NOT confirm any figure against primary sources.",
    }
    if as_json:
        from datetime import datetime, timezone
        payload["timestamp_utc"] = datetime.now(timezone.utc).isoformat()
        print(json.dumps(payload, indent=2))
    else:
        if FAILURES:
            print("=== PROVENANCE GATE: FAILED ===")
            for f in FAILURES:
                print(f)
        if WARNINGS:
            print("--- warnings (non-blocking) ---")
            for w in WARNINGS:
                print(w)
        if not FAILURES:

```

```

        print("[OK] Provenance gate passed: no stray AI artifacts, "
              "no crosswalk SSOT violation, no resurfaced excluded citations.")
        print("CAVEAT: This does not verify facts against primary sources.")
    return 1 if FAILURES else 0

def selftest():
    """Prove the gate catches what it was built to catch."""
    global ROOT # must precede any use of ROOT in this function
    fixture_root = ROOT / "tests" / "fixtures" / "contaminated"
    if not fixture_root.exists():
        print(f"[SELFTEST] Missing fixture dir: {fixture_root}")
        print(" Expected files: leaked_transcript.md, fake_crosswalk_a.py, "
              "fake_crosswalk_b.py, resurfaced_citation.txt")
        return 2
    # Temporarily retarget the scanner at the fixture tree. SKIP_DIRS
    # normally excludes "tests" (so normal runs don't scan fixtures) -
    # that same exclusion would blank out the selftest itself, so lift it
    # for the duration of this call.
    real_root = ROOT
    ROOT = fixture_root.parent.parent # ../tests
    skipped_tests = "tests" in SKIP_DIRS
    if skipped_tests:
        SKIP_DIRS.discard("tests")
    FAILURES.clear()
    WARNINGS.clear()
    try:
        scan_ai_artifacts()
        scan_crosswalk_ssot()
        scan_excluded_citations_resurfacing()
    finally:
        ROOT = real_root
        if skipped_tests:
            SKIP_DIRS.add("tests")

    required = {"[AI-ARTIFACT:]", "[DUPLICATE-SSOT]", "[EXCLUDED-CITATION-RESURFACED]"}
    tripped = {lbl for lbl in required if any(lbl in f for f in FAILURES)}
    missing = required - tripped
    if missing:
        print(f"[SELFTEST-FAIL] Gate did not trip on: {missing}")
        print("Failures actually recorded:")
        for f in FAILURES:
            print(f" {f}")
        return 1
    print("[OK] Selftest passed: all contamination patterns detected.")
    return 0

def main():
    args = sys.argv[1:]
    if "--selftest" in args:
        sys.exit(selftest())
    scan_ai_artifacts()
    scan_crosswalk_ssot()
    scan_excluded_citations_resurfacing()

```

```
sys.exit(report(as_json="--json" in args))
```

```
if __name__ == "__main__":  
    main()
```